真机上以默认配置启动、登录后，串口打印每个动态程序里 libc.so 的载入基址，这两行即证明主 ELF 与 libc.so 都被成功装载、符号被解析、重定位已完成：

```
exec: dynamic /bin/login, libc.so base=0x0000000000002000
xv6-rpi3 login: root
Password: xv6
Welcome root
exec: dynamic /bin/sh, libc.so base=0x0000000000003000
```

这里的 base 就是上文的 libbase：login 主镜像最大到 0x1a20，页对齐后落在第 2 页 0x2000；sh 更大，跨到第 3 页 0x3000。若某个程序没有打印 exec: dynamic 却能运行，说明它是静态 ET_EXEC 回退路径。

收敛与限制. 这是一个刻意为之的最小 **xv6 原生**装载 ABI，而不是完整的 ld.so：只支持 RELA（无 DT_REL）；只认上面四种动态重定位，带 GOT 数据重定位（ADR_GOT_PAGE/LD64_GOT_LO12_NC）的二进制会失败；只允许受信任的单个 /lib/libc.so，libc 自身不得再有依赖；没有 PT_INTERP、dlopen/dlsym、延迟 PLT 绑定、IFUNC、TLS、符号 interposition 与 ASLR；libc.so 每次 exec 都重新读盘并复制进本进程地址空间，进程间不共享只读物理页。于是它消除了每个程序静态携带 libc 的磁盘冗余，却还没省下运行时的物理内存。可插拔 .ko 模块的装载器与它共用 kernel/elf.h 里的数据结构，却是两套独立逻辑：模块装载器处理 ET_REL、做的是指令位域修补，而这里处理 ET_DYN、做的是把基址与符号值写回 64 位槽。

8.5.3 POSIX 层

目标是让只依赖常见 POSIX 接口的 C 源码能直接用 xv6 工具链重新编译。它由三部分组成：

1. include/ 提供标准名字的头文件：stdio.h、stdlib.h、string.h、unistd.h、fcntl.h、errno.h、time.h、pthread.h、semaphore.h、sys/types.h、sys/stat.h、sys/socket.h、netinet/in.h、arpa/inet.h、sys/epoll.h；
2. user/posix.c 把标准 API 映射到 xv6 已有的能力；
3. 内核补上无法只靠库函数实现的语义，如 lseek()，以及 socket() → bind() → listen() → accept() 的生命周期。

表 8.3: POSIX 接口到 xv6 的映射

POSIX 接口	实现
open/read/write/lseek/fstat	直接对应系统调用；O_CREAT 是 O_CREATE 的别名
getpid/getppid/getcwd/gettid	进程与当前工作目录查询
dup/dup2	复制、重定向文件描述符
wait/waitpid	回收僵尸子进程；waitpid 支持指定直接子进程、任意子进程 -1 与 WNOHANG
readlink	仅 /sys 实现（sysfs 符号链接），其它文件系统按 EINVAL 失败
pthread_create/join/exit	clone/tjoin/texit；pthread_join 同时回收用户栈
pthread_mutex_*、sem_*	内核同步对象；支持 PTHREAD_MUTEX_INITIALIZER 延迟初始化
clock_gettime(CLOCK_MONOTONIC)	clock_us()，读 CNTVCT_EL0
sleep/usleep/nanosleep	秒语义；原来按 tick 计的 sleep 改名为 sleep_ticks
socket/bind/listen/accept	统一的 socket 文件描述符，可用 read/write/close/epoll

posixtest 只包含标准头文件，验证文件读写与 lseek、getcwd、dup2、waitpid 与 getppid、POSIX 信号量、两个 pthread 并发更新共享变量、静态初始化的互斥锁以及 socket 生命周期。

这仍是第一阶段：系统调用失败只返回 −1，没有传回具体错误码，errno 也还不是线程局部的；waitpid 尚不支持进程组选择以及 WUNTRACED/WCONTINUED，WIFEXITED 暂时只对应 xv6 的普通退出状态；还缺条件变量、目录流和 TCP connect()。